

Risk Level: CRITICAL

This endpoint trusts client-supplied identifiers and marks a user's subscription active immediately after a payment attempt, without validating ownership, authentication, Stripe success state, or database lookup results. It can charge the wrong customer, upgrade arbitrary users, and leave billing/application state inconsistent.

What the AI Assumed

- The AI assumed `req.body.userId` is trustworthy and belongs to the authenticated caller.
- The AI assumed `db.users.findOne({ id: userId })` always returns a valid user object with a defined email.
- The AI assumed `db.plans.findOne({ id: planId })` always returns a valid plan object with a valid numeric price and name.
- The AI assumed `plan.price` is already in Stripe's expected smallest currency unit for amount.
- The AI assumed `paymentMethodId` can safely be used to charge this user without verifying that the payment method belongs to the customer.
- The AI assumed confirming the `PaymentIntent` with `confirm: true` means the payment fully succeeded and no further action is required.
- The AI assumed the user already has or does not need a Stripe customer record, and that charging without customer linkage is acceptable.
- The AI assumed no authorization checks are needed to determine whether the caller may upgrade the target user account.
- The AI assumed there is no need for idempotency protection, so retries will not create duplicate charges.
- The AI assumed updating `db.users`, inserting `db.auditLogs`, and sending email can happen independently without transactional consistency concerns.
- The AI assumed every paid plan change should set `subscriptionStatus` to "active" immediately, regardless of payment state, existing subscription state, or downstream failures.
- The AI assumed unhandled exceptions from Stripe, `db.users.update`, `db.auditLogs.insert`, or `email.send` will not cause problematic partial completion.

What Could Break

- An attacker can supply another user's `userId` and upgrade that account if there is no separate authentication/authorization layer enforcing ownership.
- If `user` is null, `user.email` will throw and the request may fail after a charge has already been created.
- If `plan` is null, `plan.price` and `plan.name` will throw, causing a 500 error and possible inconsistent behavior.
- If `plan.price` is stored as dollars instead of cents, Stripe may undercharge or overcharge by a factor of 100.
- A `PaymentIntent` may return `requires_action`, `requires_payment_method`, `processing`, or fail entirely, but the code still upgrades the user if `stripe.paymentIntents.create` resolves without checking `charge.status`.
- Without tying `paymentMethodId` to a Stripe customer, the API may attempt to charge an invalid, detached, or unauthorized payment method.
- Network retries or client resubmissions can create multiple `PaymentIntents` and double-charge the user because there is no idempotency key.
- If `db.users.update` succeeds but `db.auditLogs.insert` or `email.send` fails, the API leaves the plan upgraded without a corresponding audit trail or notification.

If Stripe charge succeeds but `db.users.update` fails, the customer may be charged without receiving the upgraded plan.